

Unit 2: Components, Props & State

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Functional Components: Pure JavaScript functions, JSX return, rendering components Class Components Overview: ES6 class syntax, 'React.Component', 'render()' method, legacy lifecycle Passing & Accessing Props: Props as read-only obje...

Table of Contents

1. Functional vs. Class Components	p. 2
2. Passing & Accessing Props	p. 3
3. Props vs. State (Comparative Analysis)	p. 4
4. React Fragments (<>...)	p. 5
5. Component Composition & props.children	p. 6
6. State Management with the useState Hook	p. 7
7. Immutable State Updates (Objects & Arrays)	p. 9
8. Tested Component Implementations	p. 10
9. Quick Revision Sheet	p. 12
10. Important Exam Questions & Practice	p. 13
11. Prescribed References	p. 22

1. Functional vs. Class Components

Definition:

A **React Component** is a self-contained, reusable piece of user interface that accepts inputs (props) and returns a React element tree (JSX) describing what should appear on the screen.

1.1 Functional Components (Modern Industry Standard)

A functional component is simply a JavaScript function that accepts a single props object as an argument and returns JSX. With the introduction of React Hooks (React 16.8+), functional components possess all capabilities previously restricted to classes, including local state and lifecycle side-effects:

jsx Copy

```
// Modern Functional Component
export default function WelcomeBanner({ username }) {
  return (
    <div className="banner">
      <h1>Welcome to SEM5 Portal, {username}!</h1>
    </div>
  );
}
```

1.2 Class Components (Overview & Legacy Codebases)

Before React Hooks, class components were required whenever a component needed internal state or lifecycle methods. A class component extends `React.Component` and implements a mandatory `render()` method:

jsx Copy

```
// Legacy Class Component
import React, { Component } from 'react';

class WelcomeBannerClass extends Component {
  render() {
    // Props are accessed via this.props
    return (
      <div className="banner">
        <h1>Welcome to SEM5 Portal, {this.props.username}!</h1>
      </div>
    );
  }
}

export default WelcomeBannerClass;
```

Feature	Functional Components	Class Components (Legacy)
Syntax	Plain JavaScript function.	ES6 class extending <code>React.Component</code> .
Props Access	Directly via function arguments: <code>props.name</code> or <code>{name}</code> .	Via class instance: <code>this.props.name</code> .
State Management	Using the <code>useState</code> hook.	Via <code>this.state = {...}</code> and <code>this.setState()</code> .
<code>this</code> Keyword Binding	None (no <code>this</code> confusion!).	Requires manual method binding (<code>this.handleClick.bind(this)</code>).
Code Conciseness	High (less boilerplate, easier to test and compose).	Low (verbose constructor, render method).

Quick Recall:

Functional components are plain JS functions returning JSX and managed via Hooks. Class components extend `React.Component`, access props via `this.props`, and require a `render()` method.

2. Passing & Accessing Props

Definition:

Props (short for Properties) are read-only configuration inputs passed from a parent component down to a child component, analogous to HTML attributes or function arguments.

2.1 Pure Functions & Props Immutability Rule

Fundamental Rule of React:

All React components must act like pure functions with respect to their props. A component must NEVER mutate its own props (e.g. `props.score = 100` is strictly forbidden and triggers runtime errors or unpredictability).

2.2 Destructuring Props with Default Values

Instead of typing `props.title` throughout the component body, modern React idiomatically destructures props in the function parameter list with fallback default values:

jsx

Copy

```
// Parent Component
function CourseDirectory() {
  return (
    <div>
      /* Passing different prop types: string, number, boolean, object */
      <CourseCard
        title="Front End Development using ReactJS"
        code="ISAEC594"
        credits={1}
        isElective={true}
      />
      <CourseCard
        title="Machine Learning"
        code="IS51"
        credits={4}
        isElective={false}
      />
    </div>
  );
}

// Child Component with Destructuring and Default Values
function CourseCard({ title, code, credits = 3, isElective = false }) {
  return (
    <div className="course-card">
      <h3>{title} ({code})</h3>
      <p>Credits: {credits} | Type: {isElective ? "Elective" : "Core"}</p>
    </div>
  );
}
```

Quick Recall:

Props are immutable inputs passed from parent to child. Components must never mutate props. Destructure props in parameter lists with default values: `({ title, credits = 3 })`.

3. Props vs. State (Comparative Analysis)

A classic exam question asks students to contrast Props and State across ownership, mutability, and architectural intent:

Dimension	Props (Properties)	State
Origin & Ownership	Passed from parent component (External).	Created and managed internally inside the component (Internal).
Mutability	Immutable (Read-Only) from the child's perspective.	Mutable via its dedicated setter function (setState).
Purpose	Configuring child components; passing data and event callbacks downward.	Tracking dynamic data that changes over time based on user interactions (clicks, inputs).
Re-render Trigger	Child re-renders when parent passes new prop values.	Component re-renders whenever its state setter function is called with a new value.
Analogy	Function arguments passed into a function.	Local variables declared inside the function body.

Quick Recall:

Props = external, read-only data passed from parents. State = internal, dynamic data managed by the component itself via setter functions.

4. React Fragments (<>...</>)

Definition:

A **React Fragment** allows a component to group a list of multiple adjacent children without adding extra wrapper nodes (such as unnecessary `<div>` elements) to the real DOM.

4.1 Why Are Fragments Necessary?

- **Semantic HTML Integrity:** Adding an extraneous `<div>` inside a table row (`<tr>`) or a flexbox/grid container breaks HTML validation and CSS layout flow:

jsx

```
// INVALID HTML: A <div> cannot be a direct child of <tr>!  
function TableColumns() {  
  return (  
    <div>  
      <td>IS51</td>  
      <td>Machine Learning</td>  
    </div>  
  );  
}  
  
// VALID WITH FRAGMENTS: No extra DOM node injected!  
function TableColumns() {  
  return (  
    <>  
      <td>IS51</td>  
      <td>Machine Learning</td>  
    </>  
  );  
}
```

- **Keyed Fragments:** When mapping a list of fragments inside a loop, use the explicit `<React.Fragment key={item.id}>` syntax because the short syntax (`<>`) cannot take attributes.

Quick Recall:

Fragments (`<> ... </>`) group children without inserting redundant wrapper elements into the DOM tree. Use `<React.Fragment key={id}>` when keys are required.

5. Component Composition & props.children

Definition:

props.children is a special built-in prop in React that represents whatever JSX markup or child components are nested between an opening and closing component tag.

React favors **composition over inheritance**. Instead of creating complex class inheritance hierarchies, developers create generic "box" or "container" components that receive nested content through **children**:

jsx Copy

```
// 1. Generic Container Component (Card)
function Card({ title, borderColor = "#ccc", children }) {
  return (
    <div className="card-container" style={{ border: `2px solid
${borderColor}`, padding: "16px", borderRadius: "10px" }}>
      <div className="card-header">
        <h3>{title}</h3>
      </div>
      {/* Renders whatever JSX was placed between <Card> and </Card> */}
      <div className="card-body">
        {children}
      </div>
    </div>
  );
}

// 2. Using the Card Component with Arbitrary Content
function Dashboard() {
  return (
    <Card title="Examination Alert" borderColor="#d9534f">
      <p>CIE-1 for 5th Semester begins on 20th October!</p>
      <button className="btn-alert">Download Time Table</button>
    </Card>
  );
}
```

Quick Recall:

props.children enables component composition: whatever JSX is nested inside `<Component>children here</Component>` is accessible to the component via props.children.

6. State Management with the useState Hook

Definition:

useState is the foundational React Hook that allows functional components to declare and retain local state variables across re-renders.

6.1 Hook Syntax & Array Destructuring

The `useState` function returns an array containing exactly two items:

[stateVariable, setstateVariable] = useState(initialValue);

1. `stateVariable`: The current state value during this render.
2. `setStateVariable`: A setter function used to update the state and schedule a component re-render.

6.2 Direct Updates vs. Functional (Updater) Updates

jsx Copy

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  // Direct Update: Safe when new state does not depend on current state
  const resetCount = () => {
    setCount(0);
  };

  // Functional Update: MANDATORY when new state depends on previous
  // state!
  const incrementByThree = () => {
    // If you wrote:
    // setCount(count + 1);
    // setCount(count + 1);
    // setCount(count + 1);
    // All three read the same stale snapshot of 'count' (e.g. 0),
    // resulting in count = 1!

    // PASSING AN UPDATER FUNCTION: Guarantees latest state from queue!
    setCount(prevCount => prevCount + 1); // 0 + 1 = 1
    setCount(prevCount => prevCount + 1); // 1 + 1 = 2
    setCount(prevCount => prevCount + 1); // 2 + 1 = 3 → Correct!
  };

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={incrementByThree}>+3 (Batch Safe)</button>
      <button onClick={resetCount}>Reset</button>
    </div>
  );
}
```

Quick Recall:

`const [state, setState] = useState(initial);` Always use functional updates `setState(prev  $\Rightarrow$  prev + 1)` when the new state depends on the previous state to prevent stale batching bugs.

7. Immutable State Updates (Objects & Arrays)

In React, state should be treated as **strictly read-only**. Never mutate objects or arrays in-place (e.g., `user.age = 22` or `items.push(newItem)`), because React performs an object reference identity check (`Object.is(oldState, newState)`). If the memory reference has not changed, React assumes nothing was modified and skips re-rendering!

7.1 Updating Objects via the Spread Operator (...)

jsx Copy

```
const [profile, setProfile] = useState({ name: "Kavya", semester: 5,
cgpa: 9.1 });

// WRONG: Mutates state in place! Component will NOT re-render!
// profile.cgpa = 9.3;
// setProfile(profile);

// CORRECT: Create a shallow copy and override specific fields
const updateCgpa = (newCgpa)  $\Rightarrow$  {
  setProfile(prevProfile  $\Rightarrow$  ({
    ...prevProfile,      // Copy all existing fields (name, semester)
    cgpa: newCgpa        // Override the target property
  })));
};
```

7.2 Updating Arrays Immutably

Operation	Mutating Method (DO NOT USE in React)	Immutable Idiomatic Replacement (USE THIS)
Add Item	<code>arr.push(item),</code> <code>arr.unshift(item)</code>	Spread syntax: <code>[...arr, item]</code> or <code>[item, ...arr]</code>
Remove Item	<code>arr.splice(index, 1),</code> <code>arr.pop()</code>	Filter method: <code>arr.filter(item $\Rightarrow$ item.id $\neq$ targetId)</code>

Operation	Mutating Method (DO NOT USE in React)	Immutable Idiomatic Replacement (USE THIS)
Update Item	<code>arr[index] = newItem</code>	Map method: <code>arr.map(item ⇒ item.id === targetId ? { ...item, done: true } : item)</code>
Sort / Reverse	<code>arr.sort(),</code> <code>arr.reverse()</code> (mutates original!)	Copy first: <code>[...arr].sort(...)</code>

Quick Recall:

Never mutate state in-place. Copy objects using spread `{ ...prev, field: val }`. Modify arrays using immutable methods: spread to add, `.filter()` to delete, `.map()` to update.

8. Tested Component Implementations

Complete Tested Component: Study Task Tracker with Array State:

This runnable component demonstrates functional state management, array addition via spread, and deletion via `.filter()`:

jsx Copy

```
import { useState } from 'react';

export default function StudyTracker() {
  const [tasks, setTasks] = useState([
    { id: 1, text: "Review ML Unit 1 Notes", priority: "High" },
    { id: 2, text: "Practice TOC DFA Minimization", priority:
"Medium" }
  ]);
  const [inputVal, setInputVal] = useState("");

  // Handler: Add new task immutably using spread operator
  const handleAddTask = () => {
    if (!inputVal.trim()) return;
    const newTask = {
      id: Date.now(), // Unique ID for key prop
      text: inputVal.trim(),
      priority: "Normal"
    };
    setTasks(prevTasks => [...prevTasks, newTask]);
    setInputVal(""); // Reset input field
  };

  // Handler: Remove task immutably using .filter()
  const handleDeleteTask = (idToDelete) => {
    setTasks(prevTasks => prevTasks.filter(task => task.id !==
idToDelete));
  };

  return (
    <div className="tracker-card" style={{ padding: "20px", border:
"1px solid #ddd", borderRadius: "10px" }}>
      <h3>V Semester Study Goals ({tasks.length} Pending)</h3>

      <div style={{ display: "flex", gap: "8px", marginBottom:
"16px" }}>
        <input
          type="text"
          value={inputVal}
          onChange={(e) => setInputVal(e.target.value)}
          placeholder="Enter exam study topic..."
          style={{ flexGrow: 1, padding: "8px" }}
        />
      </div>
    </div>
  );
}
```

INTERACTIVE LAB EXPLORER

Component State & Composition Studio

Tab 1: useState & Immutability

Tab 2: Props & Children Container

Explore state updates, the critical difference between direct vs. functional state updates, and immutable object modification:

A. COUNTER STATE (BATCHING & FUNCTIONAL UPDATES)

Current Count: **0**

+1 Inc

-1 Dec

Set 5

Reset

Simulate 3 Rapid Increments in 1 Handler:

3x Direct Call

3x Functional Call

Ready: Click either button to see batching behavior.

B. IMMUTABLE OBJECT STATE (SPREAD OPERATOR)

```
{  
  "name": "Priya",  
  "age": 20,  
  "enrolled": true  
}
```

Birthday (+1 Age)

Toggle Enrolled

Reset User

Shallow Ref Check: `prev !== next` (New object created → Re-render triggered)

9. Quick Revision Sheet

Concept	Syntax / Rule	Exam Trap
Functional Component	<pre>function MyComp(props) { return <div>...</div>; }</pre>	Name MUST start with a Capital letter (lowercase tags are treated as HTML!).
Props	Read-only configuration passed from parent.	NEVER mutate props inside the child component.
State	Internal dynamic data managed via <code>useState</code> .	Changing state triggers a component re-render.
Fragments	<code><> ... </></code> or <code><React.Fragment key={id}></code>	Short syntax <code><></code> CANNOT take keys or attributes.
<code>props.children</code>	Captures nested JSX between opening and closing tags.	Enables component composition rather than class inheritance.
Object State Update	<pre>setState(prev => ({ ...prev, key: val })))</pre>	Always wrap returned object literal in parentheses (<code>{ ... }</code>)!

10. Important Exam Questions & Practice

The following conceptual questions and verified code tracing problems cover essential university examination patterns for ISAEC594 (Front End Development using ReactJS):

2-Mark Conceptual Questions

 Practice Problem:

2 MARKS 1. Differentiate between Props and State in React.

Solution:

Answer:

- **Props (Properties):** External, read-only configuration passed from a parent component downward to a child. They are immutable from the child component's perspective.
- **State:** Internal, private data managed locally within a component via the `useState` hook. When state transitions occur via its setter function, React schedules a re-render of the component and its children.

 Practice Problem:

2 MARKS 2. What are React Fragments and why are they used?

Solution:

Answer:

React Fragments (`<> ... </>` or `<React.Fragment> ...`

`</React.Fragment>`) allow grouping a list of children elements without adding an extra wrapper node (such as an unwanted `<div>`) to the browser's real DOM tree.

Use Cases: Preserves semantic HTML layouts such as tables (e.g. returning multiple `<td>` elements inside a table row) and CSS Flexbox/Grid parent-child alignments.

 Practice Problem:

2 MARKS 3. Explain the purpose of `props.children` with a brief example.

Solution:

Answer:

`props.children` is a special prop available in every React component that automatically captures any JSX elements or text passed between the component's opening and closing tags.

Example: In `<Card><h2>Title</h2></Card>`, the `Card` component receives the `<h2>` node inside `props.children`, enabling flexible container composition patterns.

 Practice Problem:

2 MARKS 4. State the two fundamental Rules of Hooks in React.

Solution:**Answer:**

1. **Only Call Hooks at the Top Level:** Do not call hooks inside loops, conditional statements (`if`), or nested functions. This ensures hooks are invoked in the exact same sequential order on every render, allowing React to correctly preserve state associations.
2. **Only Call Hooks from React Functions:** Only invoke hooks from React functional components or custom hooks, never from regular JavaScript utility functions or class components.

 Practice Problem:

2 MARKS 5. Why should functional state updates `setState(prev => prev + 1)` be used instead of direct updates `setState(state + 1)`?

Solution:**Answer:**

React batches multiple state updates during event handlers and asynchronous operations for performance. Direct updates read the current render's stale closure snapshot of state. If multiple direct updates are executed in succession, subsequent calls overwrite previous ones. Passing an updater callback function `prev => prev + 1` guarantees React passes the most recent pending state to the calculation.

5-Mark & 10-Mark Model Questions

 Practice Problem:

10 MARKS 6. Explain the `useState` hook in React with syntax, mechanics, and a complete working counter implementation.

Solution:**Comprehensive Answer:**

1. **Hook Definition & Signature:** `const [state, setState] = useState(initialValue);`

`useState` is a built-in React Hook enabling functional components to maintain private, local reactive state. It takes an optional initial value and returns an array of two items using ES6 array destructuring:

- `state`: The current state variable.
- `setState`: The dispatcher function used to update the state and trigger a re-render.

2. **Update Mechanics & Batching:** When `setState` is called, React does not mutate the variable immediately in place. Instead, it enqueues a re-render request with the new value. React compares the new value with the old value using `Object.is` equality; if unchanged, React bails out of re-rendering.

3. **Complete Working Implementation:**

```
import React, { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div className="counter-card">
      <h2>Current Count: {count}</h2>
      <button onClick={() => setCount(prev => prev + 1)}>Increment</button>
      <button onClick={() => setCount(prev => Math.max(0, prev - 1))}>Decrement</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

 Practice Problem:

10 MARKS 7. Why is state immutability crucial in React? Explain how objects and arrays are immutably updated with clear code examples.

Solution:**Crucial Nature of State Immutability:**

React relies on *shallow reference comparison* (`Object.is`) to detect whether state has updated. If an object or array is mutated directly in-place (e.g. `user.age = 21` or `arr.push(item)`), the memory pointer/reference of the object remains identical. React perceives no change and will skip re-rendering the component, leading to silent UI synchronization failures.

Immutable Modification Patterns:

- **1. Updating Objects:** Always copy the existing object using the ES6 object spread operator (`...`) and override specific fields:

```
// Correct: Creates a brand new object reference
setUser(prevUser => ({
  ...prevUser,
  age: prevUser.age + 1
}));
```

- **2. Adding Items to Arrays:** Use array spread syntax instead of `push()`:

```
setItems(prev => [...prev, newItem]);
```

- **3. Deleting Items from Arrays:** Use `Array.prototype.filter()` which returns a new filtered array:

```
setItems(prev => prev.filter(item => item.id !== idToDelete));
```

- **4. Updating Items in Arrays:** Use `Array.prototype.map()` to produce a transformed array copy:

```
setItems(prev => prev.map(item =>
  item.id === targetId ? { ...item, completed: true } : item
));
```

Dimension	Functional Components (Modern)	Class Components (Legacy)
Declaration	Plain JavaScript function returning JSX	ES6 class extending <code>React.Component</code> with a <code>render()</code> method
'this' Keyword	No <code>this</code> binding required; lexical scoping throughout	Requires explicit method binding in constructor or arrow functions
State Management	<code>useState</code> and other Hooks	<code>this.state = {...}</code> in constructor; <code>this.setState()</code>
Lifecycle Logic	Unified <code>useEffect</code> hook	Partitioned methods: <code>componentDidMount</code> , <code>componentDidUpdate</code> , <code>componentWillUnmount</code>

Verified Code Tracing & Practice Problems

PRACTICE 1 · PROPS DESTRUCTURING & DEFAULTS

Trace the output of `UserBadge({ name: "Rohan", score: 95 })` when defined as: `function UserBadge({ name, role = "Student", score = 0 })`.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Consider the functional component with default parameter values:

```
function UserBadge({ name, role = "Student", score = 0 }) {  
  return {  
    badgeText: `${name} [${role.toUpperCase()}] - ${score} pts`,  
    isTopScorer: score ≥ 90  
  };  
}
```

- name is provided as "Rohan".
- role is omitted, so default value "Student" applies → `role.toUpperCase() = "STUDENT"`.
- score is 95 → `score ≥ 90` evaluates to true.
- **Result:** `badgeText: "Rohan [STUDENT] - 95 pts"`, `isTopScorer: true`.

Verification: Verified by automated test in `audit/verify/reactjs/verify_react_u2.js`.

PRACTICE 2 · PROPS.CHILDREN CONTAINER TRACE

Trace the output structure of `ModalCard` when passed title "Confirmation" and an array of 2 string children.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Function definition: `function ModalCard({ title, children }) {
 return { header: title, body: children }; }`

- `header = "Confirmation"`
- `body = ["Are you sure you want to delete this?",
 "This action is irreversible."]`
- `body.length = 2`

Conclusion: The container successfully encapsulates arbitrary child items inside `props.children` without mutating them.

Verification: Verified in

`audit/verify/reactjs/verify_react_u2.js`.

PRACTICE 3 · SEQUENTIAL USESTATE CALLS & UPDATERS

Trace the state transitions: initial value 0 $\rightarrow$ `setCount(c  $\Rightarrow$  c + 1)` $\rightarrow$ `setCount(c  $\Rightarrow$  c + 1)` $\rightarrow$ `setCount(5)`.

 Practice Problem:

Show Step-by-Step Solution

Solution:

1. Initial state: `count = 0`.
2. First functional update: `c  $\Rightarrow$  c + 1` transforms `0` $\rightarrow$ `1`.
3. Second functional update: `c  $\Rightarrow$  c + 1` transforms `1` $\rightarrow$ `2`.
4. Direct assignment: `setCount(5)` replaces state with literal `5`.
5. **Final state:** `5`.

Verification: Verified in

`audit/verify/reactjs/verify_react_u2.js`.

PRACTICE 4 · OBJECT STATE IMMUTABILITY TRACE

Given state `user = { name: "Priya", age: 20 }`, write the immutable update that increments age to 21 while preserving name.

 Practice Problem:

Show Step-by-Step Solution

Solution:

```
setUser(prev => ({
  ...prev,
  age: 21
}));
```

- The spread operator `... prev` copies all existing properties (name : "Priya").
- `age: 21` overrides the old age property.
- Outer parentheses (`{ ... }`) ensure JavaScript parses the curly braces as an object literal return rather than a function block body.

Verification: Verified in

`audit/verify/reactjs/verify_react_u2.js` (Confirms `user.age === 21` and `user.name === "Priya"`).

PRACTICE 5 · ARRAY STATE IMMUTABLE APPEND

Given state `todos = ["Read syllabus", "Write pilot"]`, show how to append "Pass exam" without mutating the original array.

 Practice Problem:

Show Step-by-Step Solution

Solution:

```
// Correct: Spread syntax creates a new array reference
setTodos(prev => [...prev, "Pass exam"]);
```

Resulting state: `["Read syllabus", "Write pilot", "Pass exam"]` (length: 3).

Anti-pattern to avoid: `todos.push("Pass exam");`
`setTodos(todos);` (Mutates in place, fails React shallow comparison!).

Verification: Verified in
`audit/verify/reactjs/verify_react_u2.js`.

PRACTICE 6 · IMMUTABLE DELETION & TOGGLE IN ARRAYS

Given task array `tasks = [{id: 1, done: false}, {id: 2, done: false}]`, write expressions to (a) delete task 1 and (b) toggle task 2 to done.

 Practice Problem:

Show Step-by-Step Solution

Solution:

- (a) Immutable Deletion:

```
setTasks(prev => prev.filter(t => t.id !== 1));
```

- (b) Immutable Field Toggle:

```
setTasks(prev => prev.map(t =>
  t.id === 2 ? { ...t, done: !t.done } : t
));
```

11. Prescribed References

Course Reference Literature:

1. Chris Minnick, *Beginning ReactJS Foundations: Building User Interfaces with ReactJS - An Approachable Guide*, Wiley, 2022. (Chapters 4, 5 & 6).
2. Anthony Accomazzo, Ari Lerner, Nate Murray, Clay Allsopp, David Gutman, and Tyler McGinnis, *Fullstack React: The Complete Guide to ReactJS and Friends*, 2017.
3. React Official Documentation: <https://react.dev>.

AI-assisted study notes written from the official courseware. Verify against the prescribed textbook before the exam.